

Guión de Presentación

Naive Bayes Híbrido

Clasificador para Variables Continuas y Binarias

Alumnos	Materia / Fecha
Ivana Noelia Zurdo	Análisis Multivariado
Nicolas Barreto	18 de junio de 2026
Tomas Muino	Prof. Patricia Duarte Lezcano
Sebastian Sanchez Bentolila	Prof. Tomas Macrade

Estructura de la presentación — 20 minutos

Sección 0 — Apertura	1 min
Sección 1 — ¿Qué es Naive Bayes?	2 min
Sección 2 — Fundamentos matemáticos	5 min
Sección 3 — Implementación en Python	5 min
Sección 4 — Tests y validación	3 min
Sección 5 — Demo en vivo y visualizaciones	3 min
Sección 6 — Conclusiones y cierre	1 min

Este documento contiene el texto completo que debe decir cada orador, junto con indicaciones de tiempo y aclaraciones para responder preguntas.

Apertura

Presentación del equipo y del trabajo

Quién habla: cualquier integrante del grupo (rotativo o el que prefieran abrir)

Lo que hay que decir — textual

Buenas tardes. Somos Ivana Zurdo, Nicolas Barreto, Tomas Muino y Sebastian Sanchez Bentalila, alumnos de Análisis Multivariado.

Hoy vamos a presentar nuestro trabajo práctico sobre el clasificador Naive Bayes. Pero no el Naive Bayes estándar que viene en las librerías: implementamos uno desde cero, en Python, que es capaz de manejar al mismo tiempo variables binarias y variables continuas. A eso lo llamamos Naive Bayes híbrido.

La presentación tiene seis partes: primero la idea general y por qué es útil, después la matemática detrás, luego el código que escribimos, los tests que hicimos para validarlo, una demo en vivo con el notebook, y por último las conclusiones.

Empezamos.

Aclaración para el orador

No apurarse. Un minuto de apertura es suficiente para presentarse y marcar la ruta.

Mirar al profesor y a los compañeros, no a la pantalla.

Respirar. La clase está prestando atención desde el primer segundo.

¿Qué es Naive Bayes?

Contexto, intuición y ventajas del clasificador

Quién habla: Integrante A

1.1 La idea en una oración

Naive Bayes es un clasificador que usa probabilidades para decidir a qué categoría pertenece un dato nuevo. Se llama 'naive' —ingenuo en inglés— porque hace una suposición muy fuerte: asume que todas las características de un dato son independientes entre sí, es decir, que cada una actúa por su cuenta sin influenciar a las demás.

Analogía para entenderlo

Imaginemos que queremos clasificar un email como SPAM o NO SPAM.

El email tiene tres características: contiene la palabra 'oferta', tiene un adjunto, y lo mandó una dirección desconocida.

Naive Bayes asume que esas tres características son independientes: que el hecho de que diga 'oferta' no tiene nada que ver con si tiene adjunto o no.

Eso no es del todo cierto en la realidad, pero funciona sorprendentemente bien en la práctica.

1.2 ¿Por qué es útil?

- **Rápido:** se entrena en una sola pasada por los datos. Sin iteraciones, sin gradientes.
- **Funciona con pocos datos:** a diferencia de redes neuronales, no necesita miles de ejemplos.
- **Interpretable:** cada parámetro tiene un significado probabilístico claro.
- **Flexible:** podemos usar una distribución diferente para cada tipo de variable.

1.3 El problema que resuelve nuestro modelo

El Naive Bayes clásico asume que todas las variables son del mismo tipo. Pero en la realidad, un dataset típico tiene mezclado de todo: altura, peso, temperatura —que son variables continuas— junto con si alguien usa barba, si compró o no algo, si hay lluvia o no —que son variables binarias.

Nuestro modelo detecta automáticamente el tipo de cada columna y aplica la distribución adecuada. A eso le llamamos clasificador híbrido.

Pregunta posible del profesor

P: ¿Cuándo falla Naive Bayes?

R: Cuando las variables están muy correlacionadas entre sí. Por ejemplo, si la altura y el peso de una persona están muy relacionados, tratarlos como independientes le da demasiado peso a

esa información (la cuenta dos veces). También puede fallar cuando la distribución real de los datos es muy diferente a Gaussiana o Bernoulli, por ejemplo si hay distribuciones multimodales o muy asimétricas.

Fundamentos matemáticos

Bayes → independencia → log → Bernoulli / Gaussiana

Quién habla: Integrante B y C se dividen esta sección (o uno solo la lleva completa)

2.1 El Teorema de Bayes — el punto de partida

Todo empieza con el Teorema de Bayes. La pregunta que queremos responder es: dado un conjunto de datos observados, ¿cuál es la probabilidad de que pertenezcan a la clase C?

$$P(C | \mathbf{x}) = P(\mathbf{x} | C) \cdot P(C) / P(\mathbf{x})$$

Donde:

$P(C)$ es el PRIOR: la probabilidad de que aparezca la clase C antes de ver ningún dato

$P(\mathbf{x} | C)$ es la VEROSIMILITUD: qué tan probable es observar estos datos si la clase fuera C

$P(\mathbf{x})$ es la EVIDENCIA: la probabilidad total de observar $\mathbf{x}$ (igual para todas las clases)

$P(C | \mathbf{x})$ es el POSTERIOR: lo que queremos, la probabilidad de C dado lo que observamos

Como $P(\mathbf{x})$ es la misma para todas las clases, podemos ignorarla al comparar. Solo nos importa el producto $P(C) \cdot P(\mathbf{x} | C)$. La clase ganadora es la que hace ese producto más grande:

$$C_pred = \operatorname{argmax}_{\text{sobre } C} \text{ de } P(C) \cdot P(\mathbf{x} | C)$$

2.2 La suposición 'naive' — independencia condicional

Si el dato $\mathbf{x}$ tiene N características —por ejemplo altura, peso y si usa barba—, la verosimilitud $P(\mathbf{x} | C)$ involucra la probabilidad conjunta de todas ellas. Calcular eso exactamente requeriría una cantidad enorme de datos.

Naive Bayes simplifica esto asumiendo que cada característica es independiente de las demás dado C . Eso transforma el producto conjunto en un simple producto de probabilidades individuales:

$$P(\mathbf{x} | C) = P(x_1 | C) \cdot P(x_2 | C) \cdot \dots \cdot P(x_N | C)$$

Ahora solo necesitamos estimar una distribución por variable y por clase, que es exactamente lo que hace nuestro código.

2.3 El problema del underflow y la solución: log-espacio

Cuando multiplicamos muchas probabilidades —todas menores a 1— el resultado se hace extremadamente pequeño. Con 50 características, podríamos estar multiplicando 50 números del tipo 0.001. Python eventualmente redondea eso a cero y pierde información. A eso se llama underflow numérico.

La solución es trabajar con logaritmos. El logaritmo transforma multiplicaciones en sumas, y trabaja perfectamente con números muy pequeños. Además, como el logaritmo es una función creciente, si un número es mayor que otro, su logaritmo también lo es —entonces el argmax no cambia:

$$\log P(C | \mathbf{x}) \propto \log P(C) + \sum_i \log P(x_i | C)$$

2.4 Distribución de Bernoulli — para variables binarias

Para variables que solo pueden valer 0 o 1, usamos la distribución de Bernoulli. Tiene un solo parámetro: p , que es la probabilidad de observar un 1.

$$p(x_i | C) = p^{x_i} * (1-p)^{(1-x_i)}$$

Estimación de p con suavizado de Laplace: $\hat{p} = (\text{cantidad de 1s en clase } C + 1) / (\text{total de muestras en } C + 2)$

El +1 en el numerador y el +2 en el denominador son los pseudoconteos de Laplace. Evitan que p sea exactamente 0 o exactamente 1.

En log-espacio: $\log(\hat{p})$ si $x = 1$ | $\log(1 - \hat{p})$ si $x = 0$

Pregunta posible: ¿por qué +1 y +2 y no otro número?

El suavizado de Laplace en su forma más simple agrega un pseudoconteo de 1 a cada valor posible.

Para Bernoulli hay dos valores posibles (0 y 1), entonces sumamos 1 al numerador (un 1 ficticio) y 2 al denominador (un 1 por cada posible valor).

Esto garantiza $0 < \hat{p} < 1$ siempre, sin importar cuántas muestras tengamos.

Con más datos, ese +1 se vuelve insignificante y el modelo aprende del dato real. Con pocos datos, actúa como una estimación segura.

2.5 Distribución Gaussiana — para variables continuas

Para variables continuas, asumimos que los datos dentro de cada clase siguen una distribución normal. Tiene dos parámetros: la media μ y la varianza σ^2 al cuadrado.

$$P(x_i | C) = (1 / \sqrt{2\pi\sigma^2}) * \exp(- (x_i - \mu)^2 / (2\sigma^2))$$

Estimación de los parámetros por máxima verosimilitud:

μ_C = promedio de los valores de x_i en las muestras de la clase C

σ^2_C = varianza de esos mismos valores

En log-espacio: $\log P(x_i | C) = -0.5 * \log(2\pi\sigma^2) - (x_i - \mu)^2 / (2\sigma^2)$

Caso especial: si $\sigma^2 = 0$ (todos los valores son iguales), usamos $\sigma^2 = 1e-9$ para evitar división por cero.

Pregunta posible: ¿por qué máxima verosimilitud?

La estimación por máxima verosimilitud (MLE) elige los parámetros que hacen que los datos observados sean lo más probables posible.

Para la distribución normal, eso resulta exactamente en la media y la varianza muestral. Es decir: el promedio de los datos y la varianza de los datos.

No hay que buscar nada más complicado. Es la estimación más natural e intuitiva.

2.6 predict_proba: convertir scores en probabilidades (log-sum-exp)

Cuando queremos probabilidades reales entre 0 y 1 (no solo el argmax), necesitamos normalizar los log-scores para que sumen 1. El truco del log-sum-exp hace esto de forma numéricamente estable:

$$P(C_k | \mathbf{x}) = \exp(s_k - \log(\sum_j \exp(s_j)))$$

Donde s_k es el log-posterior de la clase k . Restar el log de la suma antes de exponenciar evita que los $\exp()$ se disparen a infinito.

Implementación en Python

Cómo el código traduce la matemática

Quién habla: Integrante C y D se reparten los bloques de código

3.1 Estructura general de la clase

El modelo es una sola clase Python llamada NaiveBayes. Al crearse no hace nada todavía, solo inicializa cuatro variables:

- `self.classes` → las clases únicas que hay en el dataset (ej: 'hombre', 'mujer')
- `self.class_priors` → diccionario con $P(C)$ para cada clase
- `self.parameters` → diccionario con los parámetros aprendidos para cada clase y cada feature
- `self.feature_types` → lista que dice si cada columna es 'bernoulli' o 'gaussian'

3.2 Detección automática del tipo de feature

El método `_detect_feature_type` recibe una columna y mira sus valores únicos. Si todos los valores únicos están dentro del conjunto $\{0, 1\}$ —que en Python incluye True y False porque Python los trata como 1 y 0— entonces la columna es Bernoulli. En cualquier otro caso es Gaussiana.

El código, explicado línea por línea

```
unique_values = set(np.unique(column))
```

→ Calcula los valores distintos que aparecen en la columna y los mete en un conjunto.

```
if unique_values.issubset({0, 1}):
```

→ Pregunta: ¿ese conjunto es subconjunto de $\{0, 1\}$?

→ Si tiene solo 0s, solo 1s, o ambos, la respuesta es sí.

→ Si tiene un 5, un 3.14, o cualquier otro número, la respuesta es no.

```
return 'bernoulli' / return 'gaussian'
```

→ Devuelve el tipo detectado.

3.3 El método `fit()` — entrenamiento

`fit()` recibe la matriz X de features y el vector y de etiquetas y hace todo el trabajo de aprendizaje. Tiene tres pasos principales:

1. **Detectar los tipos de feature.** Llama a `_detect_feature_type` por cada columna y guarda el resultado.
2. **Calcular el prior de cada clase.** Cuántas muestras pertenecen a esa clase dividido el total. Si hay 100 muestras y 50 son 'hombre', $P(\text{hombre}) = 0.5$.

3. **Aprender los parámetros de cada feature.** Para las Bernoulli calcula p con Laplace. Para las Gaussianas calcula la media y la varianza.

Pregunta posible: ¿qué pasa si hay tres clases en vez de dos?

Nada especial. El ciclo 'for c in self.classes' itera sobre todas las clases que encuentre en y .

Si hay tres clases, el modelo aprende tres priors y tres conjuntos de parámetros.

Al predecir, calcula tres log-scores y elige el mayor. El número de clases no cambia el algoritmo.

3.4 El método predict() — predicción

predict() recibe nuevas muestras y devuelve una predicción para cada una. El proceso para cada muestra es:

- **Verificar que el modelo esté entrenado.** Si self.classes es None significa que fit() nunca fue llamado. En ese caso lanza un ValueError.
- **Para cada clase C, calcular el log-score.** Empieza con $\log(P(C))$ y va sumando el $\log P(x_i | C)$ de cada feature, usando la fórmula adecuada según el tipo.
- **Elegir la clase con mayor log-score.** Eso es el argmax. Esa es la predicción.

Nota importante para el orador

Vale la pena señalar en el notebook que la línea 'log_prob = np.log(self.class_priors[c])' inicializa el score con el prior,

y que el for interior suma los $\log P(x_i | C)$ de cada feature.

Eso muestra directamente cómo el código implementa la fórmula matemática del paso anterior.

3.5 predict_proba() — probabilidades calibradas

predict_proba() hace lo mismo que predict() hasta calcular los log-scores, pero después los normaliza para obtener probabilidades reales entre 0 y 1 que sumen 1. Usa el truco del log-sum-exp:

- Calcula los log-scores para cada clase.
- Convierte esos scores a probabilidades normalizadas usando la fórmula de softmax en espacio logarítmico.
- Retorna una matriz donde cada fila corresponde a una muestra y cada columna a una clase.

El test test_predict_proba_suma_uno verifica justamente eso: que cada fila sume 1.

Tests y validación

Por qué probamos lo que probamos

Quién habla: Integrante A o B

Escribimos 12 tests con pytest. No son tests al azar: cada uno verifica una propiedad matemática específica del modelo o un comportamiento esperado de la interfaz. Los organizamos en tres grupos:

Grupo 1 — Detección de tipos

Test	Qué verifica
<code>test_detecta_bernoulli</code>	Una columna con solo 0s y 1s se clasifica como 'bernoulli'
<code>test_detecta_gaussiana</code>	Una columna con alturas (180, 175...) se clasifica como 'gaussian'

Grupo 2 — Corrección de predicciones

Test	Qué verifica
<code>test_naive_bayes_bernoulli</code>	El modelo clasifica correctamente con solo features binarias
<code>test_naive_bayes_gaussian</code>	El modelo clasifica correctamente con solo features continuas
<code>test_predice_correctamente</code>	El modelo híbrido (Gaussiana + Bernoulli) clasifica correctamente
<code>test_predict_without_fit</code>	Si se llama predict() sin fit() primero, lanza ValueError

Grupo 3 — Propiedades matemáticas

Test	Qué propiedad matemática chequea
<code>test_laplace_evita_cero</code>	$0 < \hat{p} < 1$ siempre, aunque no haya ningún 1 en la clase
<code>test_varianza_cero_suavizada</code>	La varianza nunca es exactamente 0 (aplicamos el $1e-9$)
<code>test_predict_proba_suma_uno</code>	Las probabilidades de todas las clases por muestra suman exactamente 1
<code>test_priors_suman_uno</code>	La suma de todos los $P(C)$ es exactamente 1 (son probabilidades)
<code>test_repr_*</code>	<code>__repr__</code> devuelve 'entrenado' o 'sin entrenar' según el estado

Pregunta posible: ¿por qué np.isclose() en vez de == para comparar floats?

Los números de punto flotante tienen errores de redondeo. $0.1 + 0.2$ en Python no da exactamente 0.3 sino 0.30000000000000004.

`np.isclose()` compara con una tolerancia muy pequeña (por defecto $1e-8$), lo que permite verificar igualdad numérica sin que los errores de redondeo causen falsos negativos en el test. Si usáramos `==` el test fallaría aunque los priors sean matemáticamente correctos.

Demo en vivo y visualizaciones

El notebook en acción

Quién habla: Quien maneje el notebook (recomendamos uno que lo haya ejecutado antes)

Guión de la demo — ejecutar en este orden

1. Mostrar la celda de importaciones y ejecutar el modelo (ya en el notebook).
2. Apuntar al dataset: `X_train` tiene dos columnas. Explicar cuál es Gaussiana y cuál es Bernoulli.
3. Llamar a `fit()` y mostrar en pantalla los `feature_types` detectados.
4. Mostrar los parámetros aprendidos: para 'hombre' la altura promedio es ~177.5, para 'mujer' ~157.5.
5. Ejecutar `predict()` con los tres ejemplos de prueba y mostrar las predicciones.
6. Ejecutar `predict_proba()` y señalar que cada fila suma 1.
7. Ejecutar la celda de tests: mostrar los 12 tests pasando.
8. Mostrar los dos gráficos (distribuciones y mapa de regiones).
9. Si hay tiempo: mostrar los diagramas SVG integrados.

Lo que hay que decir en la demo

Acá lo más importante no es leer código sino mostrar qué hace el modelo. Sugerimos decir algo como:

Texto sugerido durante la demo

Como ven, le pasamos cuatro muestras al modelo: dos hombres con altura alta y barba, y dos mujeres con altura baja sin barba.

El modelo detectó automáticamente que la primera columna —altura— tiene valores continuos y la trató como Gaussiana. La segunda columna —usa barba— tiene solo ceros y unos, entonces la trató como Bernoulli.

Ahora llamamos a `predict` con nuevas muestras que el modelo nunca vio. [ejecutar] Como pueden ver, predice correctamente 'hombre' para alguien de 178cm con barba, y 'mujer' para alguien de 158cm sin barba.

En el gráfico de la izquierda ven las distribuciones Gaussianas aprendidas para la altura: la curva azul para hombres centrada en 177.5cm y la naranja para mujeres en 157.5cm. En el de la derecha ven las regiones de decisión: la zona azul es donde el modelo predice 'hombre', la naranja donde predice 'mujer'.

Pregunta posible: ¿el modelo puede equivocarse?

Sí, absolutamente. En este ejemplo el dataset es muy pequeño y perfectamente separable, entonces acierta siempre.

En datos reales hay ruido, superposición entre clases, correlaciones entre variables que el modelo ignora, y distribuciones que no son exactamente Gaussianas.

La ventaja de Naive Bayes es que incluso cuando se equivoca, da una estimación de probabilidad de cada clase, no solo una respuesta binaria.

Para medir la calidad real del modelo habría que usar cross-validation o un conjunto de test más grande.

Conclusiones y cierre

Qué aprendimos, qué hicimos, qué falta

Quién habla: Mismo que abrió, para cerrar el círculo

Lo que hay que decir — textual

Para cerrar: implementamos desde cero un clasificador Naive Bayes que funciona en la práctica.

Lo que hicimos que no es trivial es la parte híbrida: el modelo detecta automáticamente qué tipo de variable tiene cada columna y aplica la distribución matemática correcta, sin que el usuario tenga que decirle nada.

Usamos suavizado de Laplace para no tener probabilidades nulas, trabajamos en log-espacio para evitar underflow numérico, y validamos todo con 12 tests que verifican tanto el comportamiento del código como las propiedades matemáticas del modelo.

Como trabajo futuro, el modelo podría extenderse para soportar variables categóricas no binarias —con Multinomial Naive Bayes— y podría incorporarse un módulo de cross-validation para evaluar rendimiento en datasets más grandes.

Muchas gracias. Quedamos disponibles para cualquier pregunta.

Apéndice — Preguntas difíciles y sus respuestas

Esta sección es para prepararse ante cualquier pregunta adicional del profesor. No se lee en la presentación.

¿Por qué no usar `sklearn.naive_bayes` en vez de implementarlo desde cero?

Porque el objetivo del trabajo era entender e implementar el algoritmo, no usarlo como caja negra.

Además, la implementación de `sklearn` separa los tipos de Naive Bayes en clases distintas (`BernoulliNB`, `GaussianNB`, `MultinomialNB`). La nuestra los unifica en un solo clasificador con detección automática, lo que es más conveniente para datasets mixtos.

¿Cuál es la complejidad computacional del modelo?

Entrenamiento: $O(N * D)$ donde N es el número de muestras y D el número de features. Una sola pasada por los datos.

Predicción: $O(M * C * D)$ donde M es el número de muestras a predecir, C el número de clases y D las features.

Espacio: $O(C * D)$ para guardar los parámetros. Muy eficiente.

¿Qué es la varianza muestral vs. la varianza insesgada?

`np.var()` por defecto calcula la varianza con denominador N (varianza muestral / MLE).

`np.var(ddof=1)` usa denominador $N-1$ (varianza insesgada, estimador de Bessel).

Nosotros usamos la varianza MLE porque es la estimación de máxima verosimilitud, que es consistente con cómo estimamos los demás parámetros del modelo.

Con muestras pequeñas, la varianza MLE subestima ligeramente la varianza real, pero para el propósito de clasificación la diferencia es insignificante.

¿Qué es un clasificador generativo vs. discriminativo?

Naive Bayes es un clasificador GENERATIVO: modela la distribución conjunta $P(x, C) = P(x | C) * P(C)$.

Aprende cómo se 'generan' los datos dentro de cada clase.

En contraste, los clasificadores DISCRIMINATIVOS como la regresión logística o las SVM aprenden directamente la frontera de decisión $P(C | x)$ sin modelar cómo se distribuyen los datos.

¿Qué pasa si una clase tiene cero muestras en el entrenamiento?

El modelo no tendría parámetros para esa clase y lanzaría un error al predecir.

En la implementación actual, `np.unique(y)` solo devuelve las clases que aparecen en y , así que clases con cero muestras simplemente no existen para el modelo.

Si el problema requiriera manejar clases no vistas, habría que agregar tratamiento especial con priors Dirichlet o similar.

¿Por qué `True` y `False` se detectan como Bernoulli?

En Python, `True` y `False` son instancias de `bool`, que es una subclase de `int`.

Cuando numpy hace `np.unique([True, False, True])`, el resultado es `[0, 1]` porque `bool` hereda de `int`.

Por eso `set({True, False}) == {0, 1}` es `True` en Python.

Esto es un comportamiento del lenguaje que aprovechamos para que el modelo acepte columnas booleanas sin que el usuario tenga que convertirlas a enteros.

— *Fin del gui3n* —

Ivana Zurdo · Nicolas Barreto · Tomas Muino · Sebastian Sanchez Bentolila
Análisis Multivariado — 18/06/2026